

Programming and Algorithms
Aula 08: Funções em Python

Prof. Ettore Baldini-Neto
ettore.neto@faculdadeimpacta.com.br

Aula passada

- Vejam na aula 7, exemplos de strings e algumas funcionalidades.
- Formatação de strings:
  - f-string
  - exemplos
- Estruturas condicionais aninhadas e encadeadas:
  - if, else e elif

Programa da Aula

- Funções importadas
- Definindo nossas próprias funções
- Argumentos e parâmetros de funções
- Valores de retorno: return e print
- Escopo de variáveis
- Escrevendo a documentação de funções
- Organizando um código fonte

Vantagens em se trabalhar com funções

- Abstração.
  - O que é, ou o que faz e como deve ser realizado um determinado trecho de código? Respondendo a essa pergunta podemos evitar a duplicidade de comandos.
- Modularização
  - Separação do escopo: A criação de uma área de trabalho local para a função pode evitar o conflito entre variáveis internas da função e as demais variáveis de um programa.
- Lembre-se:
  - Um código legível tem interpretação e também manutenção mais fácil

Algumas perguntas

- A ideia de se construir ou utilizar funções importadas é evitar a duplicidade de alguma sequências que podem se repetir ao longo de um código.
- Perguntas importantes que queremos responder
  - Quando e por que devemos utilizar uma função?
  - Como funciona a passagem de valores para uma função?
  - Como a função pode retornar um valor de resposta?
  - O que acontece com as variáveis que criamos dentro de uma função?

Funções importadas

- O Python possui uma biblioteca padrão que tem inúmeras bibliotecas embutidas que contém um conjunto grande de **módulos** extras que são instalados mas **não são carregados automaticamente.**
- Quando precisamos usar um desses módulos temos que importá-lo para o nosso código fonte. Como fazer isso?
  - **import** <módulo> no início do código fonte.
  - normalmente se usa **import** <modulo> **as** apelido

Funções importadas

- Módulos
- Note que para acessar as funções ou constantes matemáticas -que também são chamadas de propriedades- temos que especificar o **nome do módulo e o nome da função separados por um ponto.**

```
>>> import math
>>> math
<module 'math' from '/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/lib-dynload/math.cpython-311-darwin.so'>
>>> import math as mt
>>> mt
<module 'math' from '/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/lib-dynload/math.cpython-311-darwin.so'>
>>>
```

Funções importadas

Como funciona?

- importamos o módulo math e isso cria uma variável de mesmo nome que nos dá acesso ao próprio módulo
- Ao conteúdo de um módulo chamamos de propriedades
- Para acessar as propriedades usamos um ponto (.) **após o nome da variável seguido pelo nome da propriedade que queremos usar**
- Se a propriedade for uma função precisamos de um par de parênteses para executá-la.

Funções importadas

```
>>> import math as mt
>>> mt.pi
3.141592653589793
>>> mt.log(2)
0.6931471805599453
>>> mt.sqrt(3)
1.7320508075688772
>>> mt.sqrt(3 ** 2 + 4 ** 2)
5.0
>>> mt.exp(0)
1.0
>>> mt.sin(mt.pi / 2)
1.0
>>> mt.cos(mt.pi)
-1.0
```

Outros módulos importantes em análise numérica e estatística (dados) em Python

import numpy as np-> numerical python

import pandas as pd -> analise de dados

import statistics as stat-> estatística

import scipy as sp -> ciência

Funções importadas

Exemplo: Módulo NumPy para álgebra linear, dados, etc.

Como dito, podemos utilizar apelidos para os módulos, usando o comando import < modulo> as <apelido>

```
>>> import numpy as np
>>> A = np.array([1,2,3])
>>> A
array([1, 2, 3])
>>> B = np.array([[1,4,-4],[2,3,6],[1,5,6]])
>>> B
array([[ 1,  4, -4],
       [ 2,  3,  6],
       [ 1,  5,  6]])
```

Funções definidas pelo programador

Para criar uma função, o comando é:

def <nome da função >:

___ bloco de código

Note que a indentação é a mesma do que aquela das estruturas condicionais, ou seja, quatro espaços.

Exemplo: no editor

```
def letra():
    print('Eu quis dizer, você não quis escutar')
    print('Agora não peça, não me faça promessas')
```

Funções definidas pelo programador

- **def** é uma palavra chave que indica a definição de uma função, que neste caso tem o nome de **letra.**
- Depois do nome devem vir os parênteses ( ).
- Se estão **vazio**s dizem que a função **não** usa argumentos.
- O cabeçalho da função termina com os dois pontos : que indicam o início do bloco ou corpo da função (do mesmo modo que nos comandos if e com a mesma indentação).
- Um bloco pode conter qualquer número de instruções
- Para executar o seu código no shell digite o nome da função, ou, seja letra( ).

Funções definidas pelo programador

A função é escrita no editor

```
def letra():
    print('Eu quis dizer, você não quis escutar')
    print('Agora não peça, não me faça promessas')
```

A função é executada no shell

```
>>>
    ========== RESTART: /Users/Ettore/Desktop/teste.py ==
>>> letra
    <function letra at 0x110486200>
>>> letra()
    Eu quis dizer, você não quis escutar
    Agora não peça, não me faça promessas
>>>
```


localiza a função na memória

Funções definidas pelo programador

Note: As instruções dentro da **função não são executadas até que a função seja chamada. A definição da função não gera qualquer saída.**

Fluxo de execução: Já vimos que a execução de um programa sempre começa na primeira instrução e são executadas uma após a outra. Lembre-se, no entanto, que as instruções dentro da função **não são executadas até ela ser chamada.**

Logo, uma chamada de função **causa um desvio no fluxo de execução.** Ao invés de seguir em frente o programa entra no corpo da função, executa suas instruções e volta para continuar de onde parou. E ainda uma função pode chamar outra!

Parâmetros e argumentos de funções

```
#Parâmetros de funções
def soma(n1,n2):
    s = n1 + n2
    return s
```

O retorno de um valor é feito com o comando return seguido pelo valor retornado que pode vir de uma variável ou expressão mais complexa

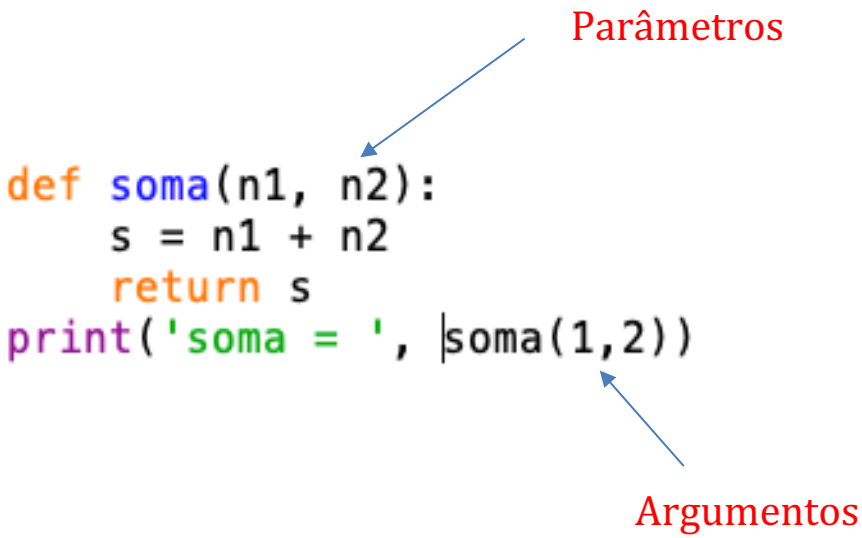
```
>>> import math as mt
>>> soma(2,3)
5
>>> soma(mt.pi,3)
6.141592653589793
>>> soma(mt.log(2),1)
1.6931471805599454
>>> soma(mt.sqrt(3 **2 + 4 **2),5)
10.0
>>> soma(mt.exp(1),1)
3.718281828459045
>>>
```

Parâmetros e argumentos de funções

Observe que a função soma abaixo tem dois parâmetros, n1 e n2.

Ao chamá-la, passamos dois valores como argumentos para os dois parâmetros que receberão esses argumentos.

Podemos entender então os parâmetros **como variáveis de entrada** da função e os argumentos como **valores que serão atribuídos a essas variáveis**.



Parâmetros e argumentos de funções

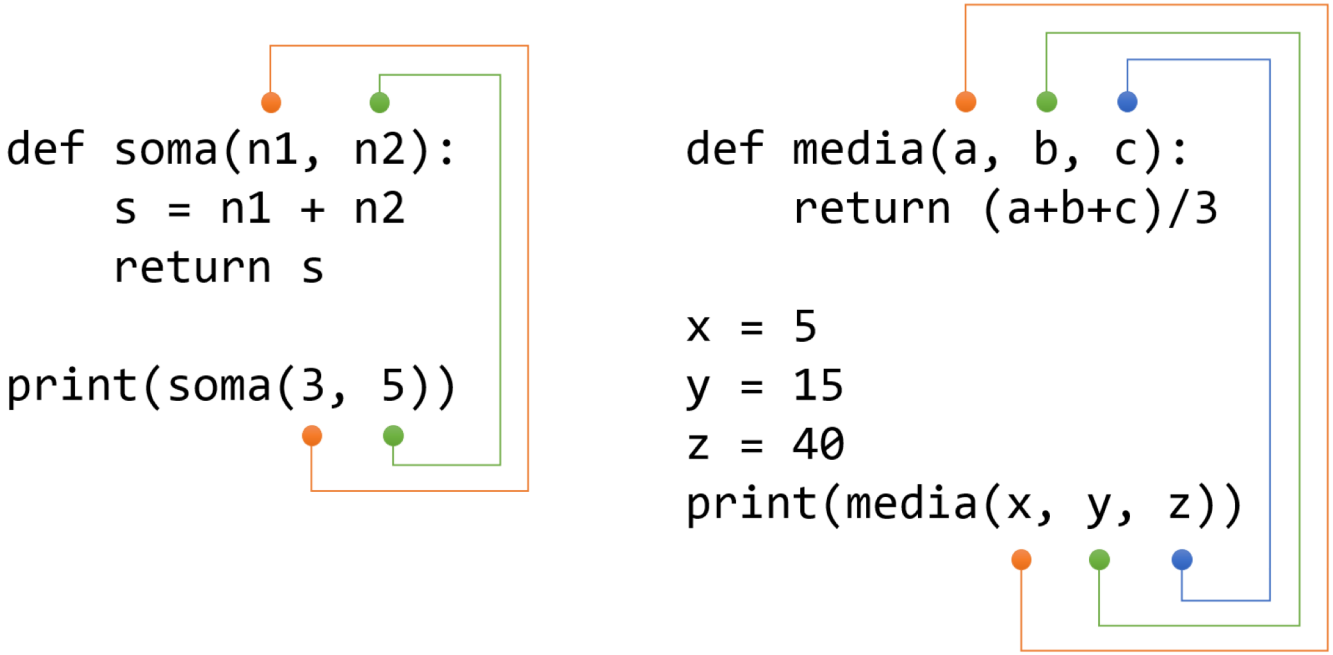


Figura 6.1: Relação entre argumentos e parâmetros. Fonte: Elaborado pelo autor.

Padrão: Os argumentos são atribuídos aos parâmetros seguindo a ordem em que são passados na chamada da função.

Fluxo de execução de uma função

```
def soma(n1, n2):
    print('Início do bloco da função')
    s = n1 + n2
    return s
    print('Fim do bloco da função')

print('Fora da função!')
```

Sem ser chamada a função não faz nada e a mensagem vem do último print.

A cada chamada, o primeiro print interno é executado, mas não o segundo pois ele está **após** o return. A função retorna o valor à direita do return (s) e **termina sua execução.** Qualquer instrução no bloco da função **posterior à execução do return é inalcançável**

No editor:

```
def soma(n1, n2):
    print('Início do bloco da função')
    s = n1 + n2
    return s
    print('Fim do bloco da função')

print('Fora da função!')
```

No shell

```
Fora da função!
>>> soma(1,2)
Início do bloco da função
3
>>> soma(-1,2)
Início do bloco da função
1
```

Sem ser chamada

Rodem no pythontutor para entender melhor

Fluxo de execução de uma função

Uma função pode ter mais de um return e isso pode ser útil caso estes returns estejam em uma estrutura de seleção que a depender do resultado usa um ou outro, mas **nunca** os dois simultaneamente.

```
def par(n):
    if n % 2 == 0:
        return True
    else:
        return False

print(f'{2} é par? {par(2)}')
print(f'{5} é par? {par(5)}')
```

Retorno versus exibição de valor

```
def soma(n1, n2):
    s = n1 + n2
    return s
```

Note que a função soma não exibe o resultado da variável s, somente retorna seu valor **quando a função é chamada**. Nem mesmo há um print na função. O retorno da função soma **só ocorre devido ao comportamento interativo do shell quando a chamamos**

Retorno versus exibição de valor

```
def soma(n1, n2):                          #1
    print('Início do bloco da função')    #2
    s = n1 + n2                            #3
    return s                               #4

print('Fora da função!')                   #5
soma(3, 5)                                 #6
soma(7, -9)                                #7
```

O que acontece quando rodamos essa codificação?
Rode no python tutor para saber!

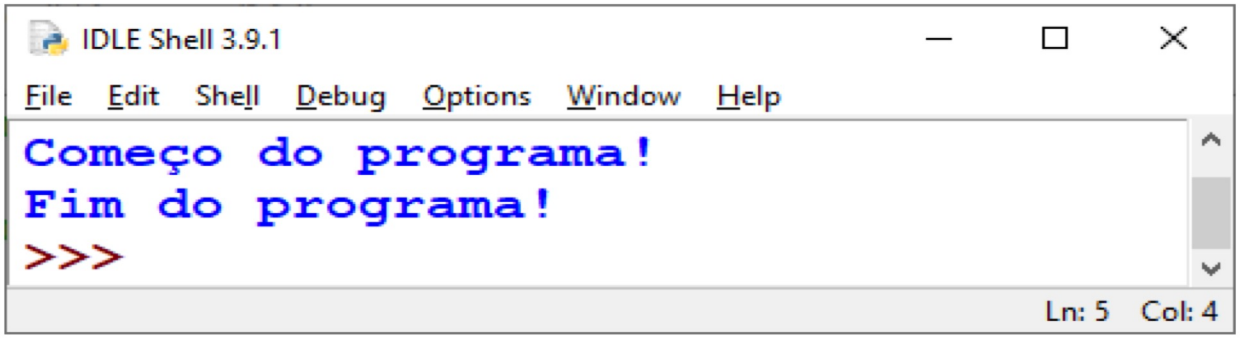
Retorno versus exibição de valor

O que acontece quando rodamos essa codificação?

Resposta: A função retornou um valor a cada chamada como era esperado, mas este valor não foi exibido.

```
print('Começo do programa!')
2 * 3 + 4
print('Fim do programa!')
```

O que acontece quando rodamos essa codificação?



Retorno versus exibição de valor

Há duas opções para que o valor final produzido por uma função seja exibido.

a) exibir o valor retornado pela função na instrução em que ela foi chamada ou atribuir o valor a uma variável para depois exibi-la

b) alterar o código de uma função para que o valor seja exibido e não retornado.

Retorno versus exibição de valor

a. função_return

```
def soma(n1, n2):
    print('Início do bloco da função')
    s = n1 + n2
    return s

print('Fora da função!')
resultado1 = soma(3,5)
print('a primeira soma é:', resultado1)
print('a segunda soma é:', soma(7,-9))
```

```
==== RESTART: /Users/Ettore/MyJob/IMPACTA/LP_2Bi/
Fora da função!
Início do bloco da função
a primeira soma é: 8
Início do bloco da função
a segunda soma é: -2
>>
```

Retorno versus exibição de valor

b. funçao_print

```
def soma(n1, n2):
    print('Início do bloco da função')
    s = n1 + n2
    print('a soma é:', s)
```

```
>>> soma(3,5)
início do bloco da função
a soma é: 8
>>> teste = soma(3,5)
início do bloco da função
a soma é: 8
>>> print(teste)
None
>>>
```

Retorno versus exibição de valor

- Em Python uma função que não executa o return, ou que o executa sem um valor à direita para ser retornado, devolve um valor especial em Python, o None (nada).
- Em outras linguagens, tentar atribuir a uma variável o valor de retorno de uma função que não retorna explicitamente algo pode gerar erro.

Quando usar print e quando usar return?

Depende: Qual o objetivo da função criada?

- Quando fazemos uma pergunta e esperamos que a função nos forneça um valor como resposta usamos **return . Esse é o caso mais utilizado.**
- **Exemplo:** Qual o resultado de uma operação qualquer entre números. Há a necessidade de retorno de valor.

- Quando damos uma ordem à função para exibir algo na tela, sem esperar uma resposta, o **print** é adequado.
- **Exemplos:** Exiba o menu de opções, mostre uma mensagem de saudação, etc. Não há necessidade de retorno de valor

Escopo de variáveis

Uma variável criada dentro de uma função é uma variável local enquanto que uma variável criada na raiz do código, ou seja, fora de uma função, é uma variável global. **Variáveis locais tem prioridade sobre variáveis globais.**

```
def diga_ola():
    print(f'Olá {nome}!')  # acessa a variável global

nome = 'Megan'             # variável global diga_ola()
diga_ola()
```

A variável nome foi criada fora da função, logo é **global.** Quando o Python procurar nome no escopo local e não encontra-la, ele vai procurar no escopo global. Caso a variável não esteja lá, ocorrerá um erro.

Escopo de variáveis

No python tutor

```
def diga_ola(nome):
    print(f'Olá {nome}!')

nome = 'Megan'
diga_ola('Maria')
print(f'Tchau {nome}!')
```

Atenção: Pode-se alterar variáveis dentro de funções para globais mas não é recomendado pois viola asregras de encapsulamento das funções, dificulta a leitura do código e suas correções.

Ver codificação a seguir:

Escopo de variáveis

Veja no python tutor

```
def diga_ola():
    global nome, titulo
    print(f'Olá {titulo} {nome}!’)
    nome = 'Megan’
    titulo = 'Dra’

titulo = 'Sra'
nome = 'Maria'
diga_ola()
print(f'Tchau {titulo} {nome}!')
```

Documentando Funções: docstrings

```
def soma(n1, n2):
    """ Retorna a soma de dois números

    Parâmetros
    ----------
    n1, n2: int ou float

    Retorno
    --------
    int ou float
        Resultado da soma de n1 com n2.
    """
    s = n1 + n2
    return s
```

```
>>> soma(
      (n1, n2)
      Retorna a soma de dois números.
```

Documentando Funções: docstrings

```
>>> help(soma)
Help on function soma in module __main__:

soma(n1, n2)
    Retorna a soma de dois números.

    Parâmetros
    ----------
    n1, n2; int ou float
         números a serem somados.

    Retorno
    -------
    int ou float
        Resultado da soma de n1 com n2
```

Organização do código fonte

```python
#importações
import math as mt

#Definição das funções
def dobro(x):
  return 2 * x

def saudacao(nome):
  print(f'Bem vindo {nome}!')

# Código principal
nome = input('Qual seu nome? ')
saudacao(nome)
dobro_pi = dobro(mt.pi)
print(f'O dobro de {mt.pi:.4f} é {dobro_pi:.4f}')
```

```
Qual seu nome? Ettore
Bem vindo Ettore!
O dobro de 3.1416 é 6.2832
```